DATA STRUCTURES (ETCCDS202)

Unit–2 Assignment Report: Linear Data Structures

Student Name	Bhavya Shany
Roll No.	2501730024
Section	A
Program	B. Tech CSE (AI & ML)
Submitted To	Mrs. Neetu Chauhan

---

1. Amortized Analysis of append() in Dynamic Array

A Dynamic Array starts with a small fixed capacity (e.g., 2 or 4). When elements are appended and the array becomes full, a new array of double the capacity is allocated, all existing elements are copied into it, and then the new element is inserted.

Why most appends are O(1)

When there is still free space inside the array, append() simply places the new element at the next available index. This is a constant-time operation — it does not matter how many elements are already present. The cost is O(1).

Why resizing is O(n)

When the array is full, every existing element must be copied one by one into the new array. If there are n elements at the time of resizing, the copy loop executes n times. This makes a single resizing event O(n).

Amortized O(1) — the intuition

Because capacity doubles each time, resizing happens very rarely — after 1, 2, 4, 8, 16, … appends. The total copy work done across all resizes to insert n elements is:

$$1 + 2 + 4 + 8 + \ldots + n \approx 2n \quad \text{(geometric series)}$$

Spreading this 2n total work over n appends gives an average (amortized) cost of 2n / n = O(1) per append. In simple words: the rare expensive resize is 'paid for' in advance by all the cheap appends that came before it.

Summary Table

Scenario	Single-call Cost	Amortized Cost
Space available	O(1)	O(1)
Array full → resize	O(n)	O(1) (amortized)
pop() from end	O(1)	O(1)

2. Linked List — Design & Complexity

A Linked List stores elements in nodes. Each node holds a value and a pointer to the next node (and, in a doubly linked list, also a pointer to the previous node). Unlike arrays, there is no contiguous memory block, so insertion and deletion do not require shifting elements.

Singly Linked List — Operations

Operation	Time Complexity	Why?
insert_at_beginning(x)	O(1)	Update head pointer only.
insert_at_end(x)	O(n)	Must traverse to last node.
delete_by_value(x)	O(n)	Must search the list first.
traverse()	O(n)	Every node is visited once.

Doubly Linked List — Extra Operations

Operation	Time Complexity	Why?
insert_after_node(target, x)	O(n)	Search for target node first.
delete_at_position(pos)	O(n)	Traverse to position pos.

Array vs Linked List — Quick Comparison

Feature	Array	Linked List
Access by index	O(1)	O(n)
Insert at start	O(n)	O(1)
Insert at end	O(1)*	O(n) / O(1) with tail
Delete by value	O(n)	O(n)
Memory	Contiguous	Scattered (extra pointer)

3. Stack & Queue — Implementation & Complexity

Stack ADT (LIFO) using Singly Linked List

The Stack is built on top of the SinglyLinkedList. Both push() and pop() operate at the head of the list, making them O(1) without any traversal.

Operation	Time Complexity	Explanation
push(x)	O(1)	New node inserted at head (insert_at_beginning).
pop()	O(1)	Head node removed and its value returned.

peek()	O(1)	Head node's value read without removal.

Underflow handling: pop() and peek() raise an exception (or print a message) when the stack is empty, preventing runtime errors.

Queue ADT (FIFO) using Singly Linked List

The Queue maintains two pointers — head (front) and tail (rear). enqueue() adds at the tail; dequeue() removes from the head. Both are O(1) because no traversal is needed.

Operation	Time Complexity	Explanation
enqueue(x)	O(1)	New node added at tail.
dequeue()	O(1)	Head node removed; head pointer updated.
front()	O(1)	Head node's value read without removal.

Underflow handling: dequeue() and front() raise an exception when the queue is empty.

Why head/tail pointers matter

Without a tail pointer, enqueue() would need to traverse the entire list to reach the last node — O(n). Keeping a dedicated tail pointer reduces this to O(1), making the Queue efficient for high-volume use cases such as task scheduling or BFS traversal.

---

4. Application — Balanced Parentheses Checker

Algorithm

1. Create an empty Stack (using our Stack ADT).
2. Scan the expression character by character.
3. If the character is an opening bracket ('(' / '{' / '['), push it onto the stack.
4. If the character is a closing bracket (')' / '}' / ']'):
   a. If the stack is empty → return False (no matching opener).
   b. Pop the top element and check whether it matches the closing bracket.
   c. If it does not match → return False.
8. After scanning all characters, return True only if the stack is empty.

Why a Stack is the right choice

Brackets must be closed in the reverse order they were opened — a Last-In First-Out (LIFO) requirement. A Stack naturally models this: the most recently opened bracket is always the first one that should be closed.

Test Cases & Expected Output

Expression	Expected Output	Reason
([])	True (Balanced)	Every bracket opened and closed correctly.
([)]	False (Not balanced)	'[' closed by ')' — mismatch.
(((	False (Not balanced)	Stack non-empty at end; unclosed brackets.
""	True (Balanced)	Empty expression; stack stays empty.

Time & Space Complexity

Metric	Value	Reason
Time Complexity	O(n)	Each character is processed exactly once.
Space Complexity	O(n)	Stack holds at most n/2 opening brackets.

---

5. Overall Complexity Summary

Data Structure	Operation	Time Complexity
Dynamic Array	append() (amortized)	O(1)
Dynamic Array	pop()	O(1)
Singly Linked List	insert_at_beginning()	O(1)
Singly Linked List	insert_at_end()	O(n)
Singly Linked List	delete_by_value()	O(n)
Singly Linked List	traverse()	O(n)
Doubly Linked List	insert_after_node()	O(n)
Doubly Linked List	delete_at_position()	O(n)
Stack	push / pop / peek	O(1)
Queue	enqueue / dequeue / front	O(1)
Parentheses Checker	is_balanced(expr)	O(n)

---

6. References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

[2] GeeksforGeeks – "Dynamic Array" https://www.geeksforgeeks.org/dynamic-array/

[3] GeeksforGeeks – "Linked List Data Structure" https://www.geeksforgeeks.org/data-structures/linked-list/

[4] GeeksforGeeks – "Stack Data Structure" https://www.geeksforgeeks.org/stack-data-structure/

[5] GeeksforGeeks – "Queue Data Structure" https://www.geeksforgeeks.org/queue-data-structure/

[6] Python Documentation – https://docs.python.org/3/

Thank You.

Efforts By-

Bhavya Shany

2501730024